1) Цель

   Изучение принципов организации непрерывной интеграции приложений с использованием облачных технологий.

2) Задание

   a) Добавить в репозиторий предыдущей лабораторной работы файл конфигурации .github/workflows/tests.yml, управляющий созданием виртуальной машины в облаке, на которой автоматически будет запускаться сборка и тестирование проекта. Примеры оформления файла конфигурации можно найти в репозиториях лабораторных работ №№1-5, в официальной документации, а также с помощью поиска в Интернете. Задачу (job) в файле конфигурации необходимо назвать test, использование других имен приведет к потере баллов. Команды в разделе steps задачи должны запускать сборку той части предыдущей лабораторной работы, которая в соответствии с вариантом была выполнена в ОС Linux (или Mac OS). Например, если по варианту задания на предыдущую лабораторную работу необходимо было разработать серверную часть приложения под ОС Linux, а клиентскую часть приложения под ОС Windows, то в данной лабораторной работе необходимо осуществить автоматическую сборку серверной части приложения. Сборка должна осуществляться упешно (без ошибок).

   b) По аналогии с предыдущей работой процесс выполнения задания должен быть разбит на задачи, которые должны быть описаны в разделе Issues репозитория. Решению каждой задачи должен соответствовать как минимум один коммит. Название задач должно начинаться с префикса Lab6:, такой же префикс должен быть в названии коммита (-ов). Коммиты и задачи без префикса засчитаны не будут.

   c) Подготовить отчет о выполнении лабораторной работы и загрузить его под именем report-lab6.pdf в репозиторий. В случае использования системы компьютерной верстки LaTeX также загрузить исходный файл report-lab6.tex.

   d) (NB! Дополнительное задание) Разработать как минимум один тест для проверки правильности функционирования приложения, автоматическая сборка которого была обеспечена на предыдущем шаге. Добавить сборку и запуск теста (-ов) в файл конфигурации tests.yml.*

   e) Согласно варианту №7:

      (i) Сервер - ОС Windows;

      (ii) Клиент - ОС Linux;

      (iii) Протокол - TCP.

      Приложение-клиент запрашивает у пользователя ввод двух строк: слово и имя текстового файла, расположенного на сервере. Эти строки передаются

на сервер, который подсчитывает количество повторений указанного слова в выбранном текстовом файле. Полученное число возвращается клиенту.

3) Описание структуры конфигурационного файла

Конфигурационный файл состоит из нескольких шагов:

a) Скопировать репозиторий;

b) Установить комплиятор G++;

c) Скомпилировать клиент файл;

d) Запустить тестирование клиента на аварийное завершение программы;

e) Скомпилировать сервер-заглушку файл;

f) Запустить тестирование на взаимодействие клиента и сервера-заглушки;

4) Содержимое написанного конфигурационного файла

tests.yml

```
name: clientbuilder
on: [push]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
      - name: Install tools
        run: |
          sudo apt-get update
          sudo apt-get install -y g++
      - name: Build client
        run:
          g++ client.cpp -o client
      - name: Test client
        run: |
          cd tests
          ./test_client.sh
      - name: Build stub server
        run:
          g++ server_stub.cpp -o server_stub
      - name: Test stub server
        run: |
```

```
cd tests
./test_server_stub.sh
```

5) Логи сборки проекта в облаке

```
Run g++ client.cpp -o client
  g++ client.cpp -o client
  shell: /usr/bin/bash -e {0}

 Run cd tests
Starting tests!
Downloading shunit2-2.1.8 ...
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed

  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
100 12958    0 12958    0     0  58196      0 --:--:-- --:--:-- --:--:-- 58107
100 52458    0 52458    0     0   197k      0 --:--:-- --:--:-- --:--:--  197k
test_clientArgs
test_clientPort
test_clientPortIsInteger

Ran 3 tests.

OK

  Run g++ server_stub.cpp -o server_stub
  g++ server_stub.cpp -o server_stub
  shell: /usr/bin/bash -e {0}

Run cd tests
Starting tests!
Found shunit2-2.1.8
test_testConnection
Сервер ожидает клиентов по localhost:25001
Введите слово для поиска:
Введите имя файла, в котором будет поиск:
Сервер получил Hi test1.txt
Слово Hi встречается в файле test1.txt в количестве 0

Ran 1 test.
```

OK

6) Исходный код тестов

client.cpp

```
// Copyright 2025 Daniil Voroshilov
#include <sys/types.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <unistd.h>
#include <iostream>
#include <string>

using std::cout;
using std::endl;
using std::string;
using std::cin;

int main(int argc, char* argv[]) {
    if (argc < 3) {
        cout << "Не указан хост или порт" << endl;
        return 1;
    }

    int port;
    try {
        port = std::stoi(argv[2]);
    } catch (std::invalid_argument& e) {
        cout << "Порт не является числом" << endl;
        return 1;
    }

    if (port < 1 || port > 65535) {
        cout << "Неверный порт" << endl;
        return 1;
    }

    // Инициализируем сокет
    int sock;
    sock = socket(AF_INET, SOCK_STREAM, 0);
    if (sock < 0) {
        cout << "Ошибка создания сокета" << endl;
```

```
    return 1;
}

// Соединяемся с сервером
sockaddr_in addr;
addr.sin_family = AF_INET;
addr.sin_port = htons(port);
if (inet_pton(AF_INET, argv[1], &addr.sin_addr) <= 0) {
    cout << "Ошибка преобразования хоста " << argv[1] << endl;
    close(sock);
    return 1;
}

if (connect(sock, reinterpret_cast<sockaddr*>(&addr), sizeof(addr)) < 0) {
    cout << "Ошибка соединения с " << argv[1] << ":" << argv[2] << endl;
    close(sock);
    return 1;
}

string Word;
string FileName;
cout << "Введите слово для поиска: " << endl;
cin >> Word;
cout << "Введите имя файла, в котором будет поиск: " << endl;
cin >> FileName;

string FullRequest = Word + ' ' + FileName;
send(sock, FullRequest.c_str(), FullRequest.size(), 0);

char ReqBuffer[512];
int recieve_bytes = recv(sock, &ReqBuffer, sizeof(ReqBuffer)-1, 0);

if (recieve_bytes > 0) {
    ReqBuffer[recieve_bytes] = '\0';
    string ServerOutput(ReqBuffer);

    cout << "Слово " << Word << " встречается в файле " <<
    FileName << " в количестве " << ServerOutput << endl;
}

close(sock);
```

```
    return 0;
}
```

server.cpp

```
// Copyright 2025 Dabiil Voroshilov

#include <winsock2.h>
#include <windows.h>
#include <sstream>
#include <fstream>
#include <string>
#include <iostream>

using std::cout;
using std::endl;
using std::ifstream;
using std::string;
using std::istringstream;

bool bisServerRunning = true;
SOCKET ListenSocket = INVALID_SOCKET;

// Функиция обработки выхода
BOOL WINAPI ConsoleHandler(DWORD signal) {
    if (signal == CTRL_C_EVENT) {
        bisServerRunning = false;  // Ручная остановка сервера
        if (ListenSocket != INVALID_SOCKET) {
            closesocket(ListenSocket);
            ListenSocket = INVALID_SOCKET;
        }
    }
    return TRUE;
}

// Функция подсчёта количества в файле
int countWordsInFile(string FileName, string InWord) {
    ifstream File(FileName);
    if (!File.is_open()) {
        return -1;
    }
```

```
    int Count = 0;
    string CurrentWord;
    while (File >> CurrentWord) {
        if (CurrentWord == InWord) {
            Count++;
        }
    }

    return Count;
}

// Функция обработки клиента по сокету
DWORD WINAPI WorkForClientThread(LPVOID lpParam) {
    // Получаем сокет
    SOCKET InSocket = *reinterpret_cast<SOCKET*>(lpParam);
    delete lpParam;

    char RecBuffer[512];
    int bytes = recv(InSocket, RecBuffer, sizeof(RecBuffer)-1, 0);
    if (bytes > 0) {
        // Получаем мьютекс по имени
        HANDLE CoutMutex = CreateMutex(NULL, FALSE, "CoutMutex");

        string Word, FileName;
        RecBuffer[bytes] = '\0';
        istringstream iss(RecBuffer);
        iss >> Word >> FileName;

        WaitForSingleObject(CoutMutex, INFINITE);
        cout << "Клиент запросил найти в файле " << FileName <<
        " количество слов " << Word << endl;
        ReleaseMutex(CoutMutex);

        int Count = countWordsInFile(FileName, Word);
        string Reply = std::to_string(Count);

        WaitForSingleObject(CoutMutex, INFINITE);
        cout << "Отправляем клиенту количество: " << Reply << endl;
        ReleaseMutex(CoutMutex);
```

```
        send(InSocket, Reply.c_str(), Reply.size(), 0);
    }

    closesocket(InSocket);

    return 0;
}

int main(int argc, char* argv[]) {
    setlocale(LC_ALL, ".utf8");
    SetConsoleCP(CP_UTF8);
    SetConsoleOutputCP(CP_UTF8);

    SetConsoleCtrlHandler(ConsoleHandler, TRUE);

    if (argc < 2) {
        cout << "Не указан порт сервера" << endl;
        return 1;
    }

    // Инициализируем WinSock
    WSADATA wsaData;
    int iResult = WSAStartup(MAKEWORD(2, 2), &wsaData);
    if ( iResult != NO_ERROR ) {
        cout << "Не удалось запустить WSAStartup()" << endl;
        return 1;
    }

    // Создаём сокет
    ListenSocket = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);

    if ( ListenSocket == INVALID_SOCKET ) {
        cout << "Ошибка создания сокета: " << WSAGetLastError() << endl;
        WSACleanup();
        return 1;
    }

    // Привязка сокета
    struct sockaddr_in service;
    service.sin_family = AF_INET;
    service.sin_addr.s_addr = inet_addr("192.168.56.1");
```

```
service.sin_port = htons(std::stoi(argv[1]));

if (bind(ListenSocket, reinterpret_cast<SOCKADDR*>(&service),
    sizeof(service) ) == SOCKET_ERROR) {
    cout << "Ошибка привязки сокета" << endl;
    closesocket(ListenSocket);
    WSACleanup();
    return 1;
}

// Прослушиваем сокет для подключений
if ( listen(ListenSocket, 1) == SOCKET_ERROR ) {
    cout << "Ошибка прослушивания сокета" << endl;
    closesocket(ListenSocket);
    WSACleanup();
}

// Клиентский сокет
SOCKET ClientAcceptSocket = SOCKET_ERROR;


// Инициализируем мьютекс
HANDLE CoutMutex = CreateMutex(NULL, FALSE, "CoutMutex");
if (CoutMutex == NULL) {
    cout << "Не удалось создать мьютекс: " << GetLastError() << endl;
    closesocket(ListenSocket);
    WSACleanup();
    return 1;
}

// Принимает клиентов
cout << "Ожидаем клиентов для подсоединения..." << endl;
while (bisServerRunning) {
    ClientAcceptSocket = accept(ListenSocket, NULL, NULL);
    if (ClientAcceptSocket == INVALID_SOCKET) {
        break;
    }

    // Обрабатываем клиента
    SOCKET* Client = new SOCKET(ClientAcceptSocket);
    CreateThread(NULL, 0, WorkForClientThread, Client, 0, NULL);
```

```
        // Идём дальше для принятия новых клиентов
        ClientAcceptSocket = SOCKET_ERROR;
    }

    WSACleanup();
    CloseHandle(CoutMutex);

    return 0;
}
```

server_stub.cpp

```
// Copyright 2025 Daniil Voroshilov
#include <sys/types.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <unistd.h>
#include <iostream>
#include <string>

using std::cout;
using std::endl;
using std::string;

int main(int argc, char* argv[]) {
    int sock = socket(AF_INET, SOCK_STREAM, 0);

    sockaddr_in addr;
    addr.sin_family = AF_INET;
    addr.sin_port = htons(25001);
    addr.sin_addr.s_addr = htonl(INADDR_LOOPBACK);

    bind(sock, reinterpret_cast<sockaddr*>(&addr), sizeof(addr));

    listen(sock, 1);

    cout << "Сервер ожидает клиентов по localhost" << ":" << 25001 << endl;

    sock = accept(sock, NULL, 0);

    char RecBuffer[512];
    int Bytes = recv(sock, RecBuffer, sizeof(RecBuffer)-1, 0);
```

```
    if (Bytes > 0) {
        string Word, FileName;
        RecBuffer[Bytes] = '\0';

        cout << "Сервер получил " << RecBuffer << endl;

        string Reply = "0";
        send(sock, Reply.c_str(), Reply.size(), 0);
    }

    close(sock);

    return 0;
}
```

7) Выводы

В процессе выполнения лабораторной работы были получены практические навыки реализации непрерывной интеграции средствами GitHub Actions. Были реализованы тесты, собирающие проект, тестирующие аварийное завершение, тестирующие взаимодействие между клиентом и сервером-заглушкой в облаке на ОС Linux.